

1541chksum

Document version: October 13th, 2025
Author Nico de Vries (Audioniek)

1541chksum is a simple programme to calculate and set the checksum bytes in (a set of) ROM(s) intended for a Commodore 1541 floppy disk drive. It is of use when developing alternative ROMs with different/added functionality. The programme currently runs only on a CBM BASIC 4.0 computer with 32 kbyte of RAM but can easily be adapted to other Commodore computers.

A 1541 floppy disk drive tests the integrity of its ROM(s) upon power up by determining its (their) checksum(s). It will not start up when the checksum(s) is/are wrong. It is therefore imperative the checksum is correct in each ROM.

The programme was written and debugged on a system consisting of a CBM 8032 computer and a 8050 dual floppy disk drive using the 121579 PET resident assembler package.

The checksum algorithm

The 1541 reset routine uses a very simple algorithm to check the checksum. All the bytes of a ROM are added together, with a carry of a previous addition taken into account when adding the next byte. The calculation is completed by adding the final potential carry by executing an addition of zero. The single byte result should match the high byte of the code address of the first byte in the ROM. At the time, this was the standard way in use at Commodore to set a ROM checksum.

In the case of the 1541 the ROM for the area \$C000-\$DFFF should have a checksum of \$C0, the ROM for \$E000-\$FFFF should have a checksum of \$E0. In each ROM, a byte is reserved to correct the checksum, in order to arrive at the desired value.

When the 1541B¹ model appeared Commodore combined the two 8 kbyte ROMs of the previous versions into one 16 kbyte device. According to the algorithm described above, the ROM should now have a single checksum spanning the full 16 kbyte and one correction byte, but Commodore did not do this. In fact, the code checking the checksum did not change, but the use of the checksum bytes did. The original checksum bytes were to remain at the values that they had before, and two new checksum correction bytes were defined.

Using the programme

The programme consists of one BASIC line and some machine code following it. It can be loaded and started as any other BASIC programme that runs on a Commodore BASIC 4.0 computer. The programme requires 32 kbyte of RAM. The programme does *not* check this, nor does it check the BASIC version it is running on.

Before loading the ROM image(s) into the area \$4000-\$7FFF, load and run the programme once to lower the top of RAM pointer. This will protect the loaded ROM contents when the programme is run the second time.

¹ Commodore's engineering department and surviving source codes call this model the 1541B, the user manual and most sources use the name 1541C. It was the last 1541 model with a built-in power supply and the only 1541 model with track zero sensor support.

The next step is to load the complete 1541 ROM image in the RAM area \$4000-\$7FFF. In case of a single 8k ROM, this will work too, provided it is located in the correct RAM area: The ROM for \$C000-\$DFFF should be loaded into the area \$4000-\$5FFF, that for \$E000-\$FFFF in the area \$6000-\$7FFF.

The checksum calculated for the area not used should then be simply ignored.

Now run the program again. After a very short while the calculated values will be shown and also inserted into the loaded ROM image. Then the programme executes a BRK, to start the MLM (machine language monitor). You can now conveniently save the ROM image on disk and program it into EPROM(s).

Before setting the checksum bytes the programme checks they are writable. If they are not, this fact is reported, and the 'READY.' prompt will appear.

Determining which checksum bytes to use is automatic, if the following is observed. The models 1541B and 1541-II have the original checksum byte for \$C000-\$DFFF at the value \$97 and that for the area \$E000-\$FFFF at \$79. If these values are already set in the image, the programme will use the alternative checksum bytes. In that case, it will show an asterisk behind the checksum value for that area. If other values, including zero, are found, the original checksum correction bytes will be used. If you want more information, see the document '1541 ROM genealogy' and/or the various original source code files. These can be found and downloaded from the internet.

Checking existing checksums is also possible. Just set/leave them in the loaded ROM image and run the programme. If the values are found to be correct, (OK) will appear behind the values for the checksum bytes.

Please note that a 1541 ROM image is not a contiguous block of bytes; it contains gaps. Commodore was in the habit of filling these areas with the value \$AA. Although the content of these areas is irrelevant, their content does affect the calculated/required checksum(s).

Building the executable

Although the programme is provided as a ready to run binary, it can be assembled using the also provided assembler source file. The source file is for the Commodore PET Resident Assembler, version 121579 and later. The source file is in clean ASCII, the line terminator is a single Carriage Return (\$0D). It can easily be adapted to an other assembler's syntax and/or computer model. Follow the manual of your assembler package to create the binary. There is only one source file; it includes the BASIC line to invoke the machine language part as .BYTE directives.

Appendix A: ROM checksum of the original 1541 ROMs

The various ROMs have the following hex checksums:

Version	C1	C2	E1	E2	Model
901229-01 + 325302-01	97	X	FD	X	1541
901229-02 + 325302-01	97	X	10	X	1541
901229-03 + 325302-01	97	X	0E	X	1541
901229-04 + 325302-01	97	X	D0	X	1541
901229-05 + 325302-01	97	X	3E	X	1541
901229-06 + 325302-01	97	X	EC	X	1541
251968-01	97	AA	79	C6	1541B
251968-02	97	46	79	CA	1541B
251968-03	97	E0	79	EB	1541-II

X means in this case: not used as checksum correction byte. That byte usually has the value \$AA in the ROM.

1541's use two 8 kbyte ROMs; the 1541B and 1541-II use one 16 kbyte device.
